

RHYTHMIX

AngularJS Login Gateway

ANGULARJS WEB DEVELOPMENT ASSIGNMENT

Field	Details
Subject	Web Technology / AngularJS Programming
Assignment Type	Login Page with AngularJS Validation & Routing
Technology Stack	HTML5, CSS3, Bootstrap 5, AngularJS 1.8.3
Submitted By	
Roll No.	
Class / Division	TE A
Date	05 March 2026
Instructor	Saiprasad Bhise

1. Aim

To build a professional Login Page for the Rhythmix Music Application using AngularJS 1.x (Angular 1), demonstrating the MVC architecture, two-way data binding, form directives, and client-side validation — which then redirects the authenticated user into the pre-built Rhythmix Vanilla JS app (index.html).

Specific learning objectives:

- Understand AngularJS module creation and dependency injection
- Apply two-way data binding using ng-model
- Handle form submission with ng-submit and the FormController object
- Use ng-show, ng-hide, ng-if, ng-class, ng-click, ng-disabled directives
- Implement non-empty field validation without a backend or database
- Pass logged-in user data to a separate Vanilla JS page via sessionStorage
- Design a visually polished, animated login UI using Bootstrap 5 and CSS3

2. Requirements

2.1 Software Requirements

Component	Technology	Purpose
Framework	AngularJS 1.8.3 (CDN)	MVC architecture, directives, data binding
CSS Framework	Bootstrap 5.3.0 (CDN)	Responsive layout, form controls, spinner
Icons	Bootstrap Icons 1.11.3 (CDN)	Eye toggle, lock, person, soundwave icons
Fonts	Google Fonts: Syne + Space Mono	Branded typography matching Rhythmix
Browser	Chrome 120+ / Firefox 118+	JavaScript ES5+ runtime
Editor	VS Code / Notepad++	Writing source files
Storage	sessionStorage (Browser API)	Pass username from login to app page

2.2 Hardware Requirements

Component	Minimum
-----------	---------

Processor	Dual-core 1.8 GHz or better
RAM	4 GB
Display	1280 x 720 or higher
Internet	Required (AngularJS loaded from CDN)

2.3 Functional Requirements

- Username field: required, non-empty, whitespace is auto-trimmed
- Password field: required, non-empty, toggleable show/hide
- Real-time validation styles applied as the user types (dirty state)
- On first submit with empty fields: all errors surface simultaneously
- On valid submit: 600ms loading spinner then redirect to index.html
- "Guest" login bypasses validation and navigates immediately
- Username stored in sessionStorage so Rhythmix greets the user by name

3. Inference / Theory

3.1 What is AngularJS?

AngularJS (Angular 1) is an open-source, front-end JavaScript framework maintained by Google. Released in 2010, it was the first JavaScript framework to popularise the MVC (Model-View-Controller) pattern for browser applications. AngularJS extends HTML with new attributes called directives and binds data to HTML using expressions. It requires no build step — a single CDN `<script>` tag is enough to bootstrap an entire app.

Key characteristics:

- Declarative templates — HTML is the view; directives describe behaviour inline
- Two-way data binding — model and view stay automatically in sync
- Dependency Injection — services like `$timeout` are injected into controllers
- MVC pattern — Model (`$scope` data), View (HTML), Controller (JS logic)
- Digest Cycle — Angular's internal loop that detects and propagates changes

3.2 AngularJS Architecture (MVC)

Layer	AngularJS Part	In This Project
Model	<code>\$scope.credentials</code>	The username and password data object
View	login.html (directives)	The form, inputs, buttons, error messages
Controller	LoginController in login.js	Handles submit, validation, redirect

3.3 Core AngularJS Directives Used

Directive	Type	What it Does	Where Used
ng-app	Structural	Bootstraps the AngularJS app on the element	<html> tag
ng-controller	Structural	Attaches LoginController to the <body>	<body> tag
ng-model	Binding	Two-way binds input value to \$scope property	Both input fields
ng-submit	Event	Calls loginUser() when form is submitted	<form> element
ng-show	DOM	Shows element when expression is truthy	Error msgs, spinner
ng-hide	DOM	Hides element when expression is truthy	Button label
ng-if	DOM	Adds/removes element from DOM	Eye icon toggle
ng-class	Style	Adds CSS class if condition is true	is-invalid / is-valid
ng-click	Event	Calls function when element is clicked	Eye btn, Guest btn
ng-disabled	Attribute	Disables element while isLoading is true	Submit button
ng-attr-type	Attribute	Dynamically sets the type attribute	Password input

3.4 Two-Way Data Binding (ng-model)

When `ng-model="credentials.username"` is placed on an input, AngularJS creates a live link between the DOM element and the `$scope` object. Any change the user types is immediately reflected in `$scope.credentials.username`. Conversely, if the controller sets `$scope.credentials.username = "Guest"`, the input field displays that value instantly without any DOM manipulation code. This bidirectional synchronisation is AngularJS's most powerful feature.

3.5 FormController and \$dirty / \$invalid

When AngularJS encounters a named `<form name="loginForm">`, it creates a FormController object and attaches it to `$scope.loginForm`. Each named input inside gets its own NgModelController accessible as `$scope.loginForm.username`. Key properties used in this project:

Property	Type	Meaning
\$dirty	Boolean	True once the user has interacted with the field
\$pristine	Boolean	True when the field has never been touched

<code>\$invalid</code>	Boolean	True when a validation rule is failing (e.g. required)
<code>\$valid</code>	Boolean	True when all validation rules pass
<code>\$error.required</code>	Boolean	True specifically when the required rule fails
<code>\$setDirty()</code>	Method	Programmatically marks the field as dirty

In the template, `ng-class` uses these properties to conditionally apply Bootstrap's `is-invalid` and `is-valid` CSS classes, giving visual feedback in real time.

3.6 sessionStorage Bridge

The login page (AngularJS) and the music app (Vanilla JS) are two separate HTML files. To pass the username across them, `sessionStorage.setItem()` is called in the controller before redirect. On the Rhythmix side, `main.js` reads `sessionStorage.getItem('rhythmix_session_user')` inside `init()` and sets `state.currentUser` accordingly. `sessionStorage` persists for the duration of the browser tab session and is automatically cleared when the tab closes.

3.7 \$timeout Service

In AngularJS, using the native `setTimeout()` inside a controller can cause changes to `$scope` to happen outside Angular's digest cycle, resulting in the view not updating. `$timeout` is Angular's wrapper — it runs code after a delay AND automatically triggers a digest cycle after completion, ensuring `$scope.isLoaded` changes are reflected in the DOM.

4. Important AngularJS Directives & Functions

Name	Category	Description
<code>angular.module()</code>	Module	Creates the AngularJS app. First argument is the app name matching <code>ng-app</code> . Second argument <code>[]</code> is the dependency list.
<code>.controller()</code>	Module API	Registers <code>LoginController</code> on the module. The array syntax <code>['\$scope', '\$timeout', fn]</code> is the safe dependency injection style.
<code>\$scope</code>	DI Service	The <code>ViewModel</code> object. Properties set here are accessible in HTML via directives. Acts as the glue between view and logic.
<code>\$timeout</code>	DI Service	Angular-safe version of <code>setTimeout</code> . Used to simulate loading delay before <code>window.location.href</code> redirect.
<code>\$scope.\$watch()</code>	Observer	Watches <code>credentials.username</code> for changes. Auto-trims whitespace. Demonstrates the observer pattern in AngularJS.

loginUser()	Controller fn	Main submit handler. Sets dirty state on all fields, checks \$invalid, sets isLoading=true, stores session, then redirects via \$timeout.
togglePassword()	Controller fn	Flips showPassword boolean. Drives ng-attr-type on the password input between text and password.
loginAsGuest()	Controller fn	Skips validation entirely. Stores Guest in sessionStorage and navigates to index.html after 400ms.
loginForm.\$invalid	FormController	Boolean set by AngularJS based on all required fields. If any required field is empty, this is true.
loginForm.username.\$setDirty()	FormController	Forces the field into dirty state on first submit so ng-class validation styles appear even without typing.
ng-class expression	Directive	Object syntax: {'is-invalid': condition} — adds the class when condition is truthy. Enables Bootstrap validation styles.
sessionStorage.setItem()	Web API	Stores the username string in browser session storage for consumption by the Rhythmix main.js init() function.

5. File Structure

All files must be in the same folder for relative paths to work:

File	Role	Key Concepts
login.html	AngularJS view — the login page	ng-app, ng-controller, ng-model, ng-submit, ng-show, ng-class, ng-click, ng-if, ng-attr-type
login.css	Login page stylesheet	CSS variables, animations, Bootstrap overrides, dark theme
login.js	AngularJS module + controller	angular.module, .controller, \$scope, \$timeout, \$watch, sessionStorage
index.html	Rhythmix app (pre-built)	The destination after login — reads sessionStorage
style.css	Rhythmix app styles	Custom CSS for music player UI
main.js	Rhythmix Vanilla JS	Reads sessionStorage to auto-login the Angular user

Flow: User opens login.html → fills fields → AngularJS validates → on success, stores username in sessionStorage → redirects to index.html → main.js reads sessionStorage → Rhythmix greets user by name.

6. Program / Implementation

6.1 login.html — Structure

- Loads AngularJS from CDN before login.js — order matters
- `ng-app="rhythmixApp"` on `<html>` bootstraps the entire app
- `ng-controller="LoginController"` on `<body>` creates the `$scope`
- Form has `name="loginForm"` — exposes `FormController` to `$scope`
- `novalidate` on form — disables browser popup bubbles, lets Angular handle validation
- Each input has `name`, `ng-model`, `ng-class`, and `required` attributes
- Error `<div>`s use `ng-show` with `$dirty && $error.required` conditions
- Submit button uses `ng-disabled="isLoading"` to prevent double-submit
- Password input uses `ng-attr-type="{{ showPassword ? 'text' : 'password' }}"`

6.2 login.css — Design Decisions

- CSS custom properties on `:root` for consistent dark theme tokens
- Animated radial gradient blobs in `.bg-particles` using `@keyframes floatUp`
- Login card entrance animation: `translateY` + `scale` cubic-bezier for "pop" effect
- Pulsing glow on brand icon using `box-shadow` animation
- `.is-invalid` and `.is-valid` override Bootstrap defaults for dark background compatibility
- Mobile responsive: `@media (max-width: 480px)` collapses padding

6.3 login.js — Controller Logic

- Module declared with `var app = angular.module('rhythmixApp', [])`
- Controller uses array injection syntax for minification safety
- `$scope.credentials` object is the single source of truth for form data
- `loginUser()` calls `$setDirty()` on each field before checking `$invalid` — this ensures errors show on first submit even if user never touched the fields
- `$timeout` wraps the redirect to guarantee Angular's digest cycle processes `isLoading=true` before navigation
- `$scope.$watch` on `username` demonstrates the observer pattern used frequently in real AngularJS apps

7. Expected Output

Test Scenario	Expected Result
Open login.html	Animated card appears with brand logo, two input fields, login button, guest option
Click "Enter Rhythmix" with both fields empty	Both fields turn red with error messages. Alert banner appears.

Type in username only, click submit	Username turns green. Password turns red with error. Submit blocked.
Type spaces in username field	\$watch trims spaces; treated as empty; validation fails
Fill both fields, click submit	Spinner appears, button disabled for 600ms, then redirected to index.html
index.html loads after login	Welcome alert shows "Welcome to Rhythmix, [username]!" sidebar shows user name
Click "Guest — Skip Login"	Redirected immediately to index.html as "Guest"
Click eye icon on password	Input switches to plain text. Icon changes to eye-slash.
Hover submit button	Button lifts with shadow glow in accent lime color

8. Conclusion

This assignment successfully demonstrates AngularJS 1.x as a structured, declarative alternative to plain Vanilla JavaScript for building interactive web forms. The login page built for Rhythmix illustrates the following core Angular concepts in a practical, real-world context:

- `angular.module` and `.controller` for modular app structure
- `$scope` as the ViewModel bridge between HTML and JavaScript logic
- `ng-model` providing automatic two-way data binding with zero DOM code
- `FormController` (`$dirty`, `$invalid`, `$setDirty`) for declarative form validation
- `ng-class`, `ng-show`, `ng-if` for dynamic rendering driven entirely by `$scope` state
- `$timeout` for safe asynchronous operations within Angular's digest cycle
- `sessionStorage` as a lightweight cross-page data bridge without a backend

The project demonstrates that even a simple login form benefits from AngularJS's declarative approach — validation logic in the controller stays clean, the HTML template is self-documenting, and extending the form (adding more fields, rules) requires minimal code changes. This sets the foundation for understanding modern component-based frameworks like Angular 2+, React, and Vue.

9. References

1. AngularJS Official Documentation: <https://docs.angularjs.org/api>
2. AngularJS Developer Guide — Forms: <https://docs.angularjs.org/guide/forms>
3. AngularJS CDN (Google):
<https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js>
4. Bootstrap 5 Documentation: <https://getbootstrap.com/docs/5.3/>
5. MDN Web Docs — `sessionStorage`: <https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage>

6. W3Schools AngularJS Tutorial: <https://www.w3schools.com/angular/>
7. Bootstrap Icons: <https://icons.getbootstrap.com/>

— End of Assignment Document —